

Continuous-Time Latent Dynamics for JEPA World Models: ODE Prediction in LeWorldModel

Boyi Li^a

^aIndependent / AutoConference

Abstract. Joint Embedding Predictive Architecture (JEPA) world models such as LeWorldModel (LeWM) learn compact latent dynamics for model-based control, typically with a discrete autoregressive (AR) next-embedding predictor. We replace that predictor with an action-conditioned neural ODE $dz/d\tau = v_\theta(z, a)$, where one planning step is a fixed-step RK4 integration from $\tau=0$ to $\tau=1$. Encoder, SIGReg anti-collapse regularizer, projectors, and the CEM planning loop are unchanged, isolating the effect of continuous latent dynamics. On the official PushT expert dataset with a fixed 1000-window subset (seed 3072; train 901 / val 99), both the discrete AR baseline and our ODE predictor achieve the same CEM success of **5/93** ($\approx 5.38\%$) on the val-tied protocol, while the ODE model attains lower train and validation losses (0.234 / 0.506 vs. 0.248 / 0.536). We report honest limitations: the 1k subset is far below the paper-scale regime that reports $\approx 96\%$ PushT success, and CEM is tied despite the loss improvement. All data are from the official Hugging Face release; no synthetic episodes are used.

1 Introduction

1.1 Motivation

World models that plan in a compact latent space enable fast model-predictive control without decoding pixels at every step [6, 7]. Classical latent dynamics models often predict the next state with a discrete transition (RNN, RSSM, or transformer block), then hand the rollout to a planner such as CEM or MPPI. Joint Embedding Predictive Architectures (JEPAs) [1] push the representation side further by predicting future *embeddings* rather than reconstructions, avoiding a pixel decoder that can dominate compute and encourage short-cut features. LeWorldModel (LeWM) [9] shows that a stable end-to-end JEPA can be trained from pixels with only two loss terms: next-embedding MSE and SIGReg isotropic-Gaussian regularization [2]. The resulting system supports Cross-Entropy Method (CEM) planning directly in latent space and reports strong PushT success under full-data training.

In LeWM, the predictor is a discrete autoregressive transformer block: given context latents z_t and action embeddings a_t , it directly outputs $\hat{z}_{t+1}$. Physical dynamics, however, are continuous in time. A discrete next-step map must absorb the entire transition into a single nonlinear layer, with no explicit notion of intermediate flow along the planning interval. Recent continuous-time world models such as ODEWorld [8] argue that latent velocity fields $dz/dt = v_\theta(\cdot)$ better capture underlying flow and support flexible temporal resolution.

Neural ODEs [4] provide the standard tool for parameterizing such fields and integrating them with classical solvers.

This creates a natural inductive-bias question for planning JEPA models: if the rest of the LeWM stack—encoder, SIGReg, projectors, CEM—is held fixed, does replacing the discrete AR predictor with an integrate-to-next-step ODE improve representation quality and/or planning success on official control data?

1.2 Problem statement

We study a *minimal* predictor swap under a matched experimental protocol. Let f_ϕ be the pixel encoder, p and q the projectors, $\mathcal{L}_{\text{SIGReg}}$ the anti-collapse regularizer, and CEM the official planner. The discrete baseline predicts

$$\hat{Z}_{t+1} = g_\theta^{\text{AR}}(Z_t, A_t), \quad (1)$$

while our continuous alternative defines a velocity field v_θ and

$$\hat{Z}_{t+1} = \text{ODESolve}(v_\theta; Z_t, A_t; \tau:0 \rightarrow 1). \quad (2)$$

Both models are trained with the same JEPA objective $\mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{SIGReg}}$ on the same official PushT windows, and evaluated with the same val-tied CEM starts. We deliberately do *not* redesign the encoder, change SIGReg hyperparameters, introduce pixel reconstruction, or invent new planning costs: any difference is attributable to the predictor dynamics.

Can we replace LeWM’s discrete AR predictor with an ODE integrate-to-next-step rule, keep the rest of the pipeline fixed, and retain competitive CEM planning on official PushT data?

1.3 Contributions

1. **ODE drop-in predictor.** We instantiate an action-conditioned ODE predictor inside the official LeWM JEPA: $dz/d\tau = v_\theta(z, a)$, with one-step prediction defined as classical RK4 integration over $\tau \in [0, 1]$ (fixed $N=4$ substeps), actions held constant on the interval, and a zero-initialized linear velocity head for residual stability at initialization. The velocity backbone reuses the same ConditionalBlock / AdaLN-zero transformer capacity as the discrete AR predictor ($\approx 18.07\text{M}$ parameters; +37,056 from the v -head).

2. **Matched 1k protocol on official data.** We evaluate on the official Hugging Face PushT expert HDF5 with a reproducible 1000-window split (seed 3072; 901 train / 99 val), matching encoder, SIGReg ($\lambda=0.09$, $M=1024$), optimizer, hardware batching ($64\times$ accum 2), and CEM protocol between discrete and ODE runs. Indices are frozen and never regenerated between runs.
3. **Honest empirical findings.** On this protocol, ODE lowers train/val loss relative to discrete AR (0.234 / 0.506 vs. 0.248 / 0.536), while CEM success on the val-99 protocol is **tied** at 5/93 (5.38%). We explicitly contrast this with the paper-reported $96\pm 2.83\%$ PushT figure under full-data training, and treat the 1k regime as a controlled ablation rather than a claim of paper-scale performance. No synthetic episodes are used.

Why this matters. If continuous dynamics help even as a drop-in, JEPA world models can borrow ODE inductive bias without abandoning SIGReg or CEM. If they do not help planning under matched compute and data, that is also useful: it suggests that predictor form is secondary to data scale and planner configuration in the 1k regime. Our results land in the middle—loss improves, CEM does not—and we report both sides without overselling.

Scope. This paper is a controlled predictor ablation, not a full re-training of LeWM at paper scale, and not a multi-environment bake-off. Cube, Two-rooms, and Reacher remain future work (Section 4.3). We also do not convert or evaluate the official Hugging Face pretrained PushT weights under our val-99 protocol; that baseline remains future work.

2 Related Work

2.1 JEPA, SIGReg, and LeWM

Joint Embedding Predictive Architectures learn predictive representations by forecasting future embeddings rather than pixels [1]. The JEPA objective is typically a distance between a predicted latent and a target latent from a context encoder, often with stop-gradient on the target path so the predictor cannot collapse by copying a degenerate constant. Video JEPA variants extend this idea to temporal sequences [3], still avoiding reconstruction decoders that can dominate loss and encourage shortcut visual features. A recurring failure mode is *representation collapse*: embeddings that ignore inputs yet trivially satisfy a prediction objective. SIGReg [2] addresses collapse by pushing projected embedding coordinates toward an isotropic Gaussian via a sketched Epps–Pulley statistic, without requiring a stop-gradient teacher network as in contrastive or BYOL-style recipes.

LeWM [9] brings JEPA to end-to-end pixel world models for control. Its recipe is deliberately simple: a ViT encoder trained from scratch, an action-conditioned predictor, MSE next-embedding loss, and SIGReg with weight $\lambda=0.09$ and $M=1024$ projections. Planning uses CEM in latent space against a goal embedding derived from a future observation. Reported PushT success under full-data training is high ($96\pm 2.83\%$), which motivates careful protocol matching when studying predictor variants on smaller subsets: a weak absolute CEM score on 1k windows is expected and should not be confused with a failed reproduction of the full recipe. We adopt LeWM’s official encoder, SIGReg recipe, projectors, and CEM evaluator unchanged, so our study isolates predictor dynamics rather than re-tuning the full stack.

2.2 Continuous-time and ODE world models

Neural ODEs [4] parameterize continuous transformations via $dz/dt = v_\theta(z, t)$ and backpropagate through ODE solvers. This viewpoint has influenced continuous-depth networks, continuous normalizing flows, and—more recently—world models that treat latent evolution as a flow rather than a discrete jump.

ODEWorld [8] introduces Physical-Time Flow: a continuous latent velocity field integrated by an ODE solver for video and control, emphasizing dynamical representation structure beyond discrete next-step prediction. Our setting is narrower and deliberately comparable: we keep LeWM’s JEPA loss and planning loop, and redefine a single discrete planning step as integrate $\tau:0 \rightarrow 1$ under an action-conditioned velocity field. We do not claim the full Physical-Time Flow agenda (adaptive physical time, multi-scale horizons, or ODE-World’s broader architecture); instead we ask whether the ODE inductive bias alone helps under a matched LeWM protocol.

2.3 Discrete AR vs. continuous latent dynamics

Discrete autoregressive predictors map $(z_t, a_t) \mapsto \hat{z}_{t+1}$ in one shot. They are efficient at inference (one forward pass per step) and match the discrete action schedule used by CEM rollouts. Continuous predictors instead learn v_θ and obtain $\hat{z}_{t+1}$ by numerical integration. This costs more compute per step (RK4 with $N=4$ evaluates the velocity field four times per substep interval, sixteen times over $\tau:0 \rightarrow 1$ with $N=4$), but exposes intermediate states along the interval and encourages residual updates ($z(1) \approx z(0)$ when $v \approx 0$). Our experiments quantify both sides: loss and CEM on a fixed split, plus wall-clock CEM cost on the same hardware.

2.4 Latent MPC and CEM planning

Model-based RL and planning systems often learn latent dynamics and optimize action sequences by shooting [5, 7]. Cross-entropy method (CEM) samples candidate action sequences, rolls out the learned dynamics, ranks by a goal-conditioned cost, and refits a sampling distribution around elite samples. LeWM uses this interface with a latent goal embedding derived from a future observation at a fixed goal offset.

We use the official LeWM CEM evaluator on a fixed validation window set so discrete and ODE predictors share the same starts and feasibility filter (Section 4.1). Earlier probes on random full-dataset starts are discarded as protocol-mismatched; all planning claims in this paper use the val-99 / 93-feasible protocol tied to `validate/loss`.

2.5 Positioning summary

Relative to prior work, ODE-LeWM is intentionally conservative:

- vs. LeWM: same JEPA + SIGReg + CEM stack; only the predictor map changes from discrete AR to RK4-integrated v_θ .
- vs. ODEWorld: we borrow the continuous-velocity idea, but do not adopt Physical-Time Flow, adaptive physical time, or ODE-World’s full training recipe.
- vs. classical Neural ODEs: we integrate on a fixed normalized interval $\tau \in [0, 1]$ per planning step, with piecewise-constant actions, rather than learning an unrestricted continuous-time policy.

This positioning keeps comparisons fair on official PushT windows and makes negative or tied CEM results interpretable rather than confounded by simultaneous encoder or planner changes.

3 Method

We keep the LeWM JEPA backbone [9] and replace only the discrete AR predictor with an ODE velocity field. Notation follows the official implementation (history length $H=3$, embedding dimension $D=192$, number of predictions per window 1).

3.1 Encoder and action embedding

Let $x_{t-H+1:t}$ be a stack of RGB observations at resolution 224×224 . A ViT-tiny encoder f_ϕ (patch size 14, 12 layers, hidden size 192, 3 attention heads, trained from scratch) maps each frame to a CLS token. An MLP projector p with BatchNorm yields latents

$$z_t = p(f_\phi(x_t)) \in \mathbb{R}^D. \quad (3)$$

Raw actions u_t (PushT end-effector commands) are mapped by an embedder e_ψ to

$$a_t = e_\psi(u_t) \in \mathbb{R}^D. \quad (4)$$

Context tensors for one prediction step are the history windows

$$Z = (z_{t-H+1}, \dots, z_t), \quad A = (a_{t-H+1}, \dots, a_t), \quad (5)$$

each of shape (H, D) per batch element (implementation uses (B, H, D)).

3.2 Discrete baseline predictor

The official LeWM predictor is an action-conditioned transformer (ConditionalBlock / AdaLN-zero) of depth 6, 16 heads, width D , MLP dimension 2048, and dropout 0.1:

$$\hat{Z}_{t+1} = g_\theta^{\text{AR}}(Z, A), \quad (6)$$

interpreted as the next latent window (same shape as Z). Internally, learnable positional embeddings are added along the history axis, and actions A condition every block through AdaLN. A prediction-head MLP q is applied after the predictor for the MSE term; q is unchanged in our ODE variant.

3.3 ODE predictor

We reinterpret the same transformer capacity as a **velocity field** v_θ , followed by a zero-initialized linear head $W_v \in \mathbb{R}^{D \times D}$ with bias (adds $D^2 + D = 37,056$ parameters):

$$v_\theta(z, a) = W_v \text{TF}_\theta(z + \text{PE}, a), \quad (7)$$

where PE is a learnable positional embedding over the history axis and actions a condition AdaLN as in the AR baseline. Latent dynamics on a normalized step interval $\tau \in [0, 1]$ are

$$\frac{dz}{d\tau} = v_\theta(z(\tau), a), \quad z(0) = Z, \quad (8)$$

with a held fixed over the interval (piecewise-constant action, matching one CEM planning step and the discrete action schedule). **One-step prediction** is the solution at $\tau=1$:

$$\hat{Z}_{t+1} = z(1) = Z + \int_0^1 v_\theta(z(\tau), a) d\tau. \quad (9)$$

We approximate the integral with fixed-step classical RK4 using $N=4$ substeps ($h=1/N$); no `torchdiffeq` dependency is required. One RK4 substep is

$$\begin{aligned} k_1 &= v_\theta(z, a), \\ k_2 &= v_\theta(z + \frac{h}{2}k_1, a), \\ k_3 &= v_\theta(z + \frac{h}{2}k_2, a), \\ k_4 &= v_\theta(z + h k_3, a), \\ z &\leftarrow z + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \end{aligned} \quad (10)$$

repeated N times from $z(0) = Z$ to obtain $z(1)$. Zero-init of W_v yields $v \approx 0$ at initialization, hence $z(1) \approx z(0)$ (stable residual ODE), analogous to AdaLN-zero initialization in the discrete predictor.

Remark 1 (Discrete interface, continuous internals). *The planner still sees a discrete next-window map $Z \mapsto \hat{Z}_{t+1}$. Continuity lives inside the map via RK4; we do not change CEM’s horizon, action blocking, or cost.*

3.4 Training objective (JEPA + SIGReg)

Following LeWM, the total loss is

$$\mathcal{L} = \mathcal{L}_{\text{JEPA}} + \lambda \mathcal{L}_{\text{SIGReg}}, \quad \lambda = 0.09, \quad (11)$$

where we write $\mathcal{L}_{\text{JEPA}}$ for the next-embedding prediction term (also called $\mathcal{L}_{\text{pred}}$ in logs). Concretely, if q is the prediction projector and stop-grad targets are $\text{sg}[p(f_\phi(x_{t+1}))]$ (history-aligned),

$$\mathcal{L}_{\text{JEPA}} = \|q(\hat{Z}_{t+1}) - \text{sg}[p(f_\phi(x^{\text{tgt}}))]\|_2^2, \quad (12)$$

and $\mathcal{L}_{\text{SIGReg}}$ is the sketched Epps–Pulley statistic over $M=1024$ random projections and 17 quadrature knots [2, 9]. Writing P for projected embedding coordinates and t_k for quadrature nodes, SIGReg accumulates a discrepancy between the empirical characteristic function of P and that of $\mathcal{N}(0, 1)$ across random directions; we use the official LeWM implementation unchanged (weight $\lambda=0.09$). Only the predictor internals differ between discrete and ODE runs; encoder, projectors, SIGReg, and optimizer are shared.

3.5 Planning interface (CEM)

At evaluation, Cross-Entropy Method (CEM) samples action sequences, rolls out the predictor in latent space from encoded pixels, and selects the plan minimizing distance to a goal embedding (official PushT CEM config: 300 samples, 30 CEM iterations, top- $k=30$, horizon / receding horizon / action block = 5, goal offset 25 steps, max episode length 100). For the ODE model, each rollout step applies (9)–(10) instead of (6). Encoder and goal embedding construction are identical.

3.6 Implementation notes

The ODE predictor is implemented as a drop-in module with the same $(x, c) \rightarrow \text{pred}$ signature as the discrete AR predictor, so the JEPA forward path, SIGReg hook, and CEM rollout loop need no API changes beyond selecting the predictor class in the model config. We implement fixed-step RK4 (and Euler) in-repo; training runs use RK4 with $N=4$. Velocity-head zero-init is applied once at construction. On the RTX 4060 Laptop (8 GiB), memory pressure

Algorithm 1 ODE one-step prediction (training & CEM rollout)

Require: Context latents Z , action embeddings A , steps $N=4$

- 1: $z \leftarrow Z$; $h \leftarrow 1/N$
- 2: **for** $i = 1$ to N **do**
- 3: $k_1 \leftarrow v_\theta(z, A)$
- 4: $k_2 \leftarrow v_\theta(z + \frac{h}{2}k_1, A)$
- 5: $k_3 \leftarrow v_\theta(z + \frac{h}{2}k_2, A)$
- 6: $k_4 \leftarrow v_\theta(z + h k_3, A)$
- 7: $z \leftarrow z + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
- 8: **end for**
- 9: **return** $\hat{Z}_{t+1} \leftarrow z$ $\triangleright z(1)$ after $\tau:0 \rightarrow 1$

Algorithm 2 Matched train / eval protocol (discrete vs. ODE)

- 1: Draw 1000 PushT windows with seed 3072; split 901/99
- 2: Train JEPa with $\mathcal{L}_{\text{JEPa}} + \lambda \text{SIGReg}$ for 100 epochs (AdamW, bf16, effective batch 128)
- 3: Select epoch-100 checkpoint
- 4: CEM on 93 feasible val windows (goal offset 25)
- 5: Report fit/loss, val/loss, and CEM successes / 93

forces `num_workers=0` and the $64 \times \text{accum-2}$ batch recipe for both arms of the comparison, so wall-clock differences at train time are dominated by the extra velocity evaluations inside RK4 rather than by batch-size mismatch.

4 Experiments

4.1 Setup

Data. We use only the official Hugging Face PushT expert release (quentinll/lewm-pusht, HDF5; no Cube / Two-rooms / Reacher downloads in this study). A reproducible subset of **1000** windows is drawn with seed **3072** and split by `spt.random_split([0.9, 0.1])` into **901** train / **99** validation windows. Because the validation fraction is passed as $1 - 0.9$ in floating point, $\lfloor 1000 \times 0.1 \rfloor$ yields 99 validation indices and the remainder assigns 901 to train—matching the frozen subset files (pusht_expert_train_n1000_seed3072 train/val index arrays). Indices are never regenerated between discrete and ODE runs. Of the 99 val windows, **93** remain feasible under goal offset 25 for CEM (6 dropped because the goal frame would fall past the episode end).

Models and compute. Both predictors use ViT-tiny, history 3, $D=192$, SIGReg $\lambda=0.09$, AdamW lr 5×10^{-5} , weight decay 10^{-3} , grad clip 1.0, bf16, 100 epochs, frameskip 5. Hardware: RTX 4060 Laptop (8 GiB). Official batch size 128 OOMs with dataloader workers on this GPU; we run batch 64 with gradient accumulation 2 (effective 128) and `num_workers=0`. Parameter counts: discrete AR **18,034,478**; ODE **18,071,534** (+37,056 from the velocity head).

4.2 Results

Table 3 summarizes the matched comparison. The ODE predictor improves fit/loss (0.234 vs. 0.248) and val/loss (0.506 vs. 0.536); final val prediction / SIGReg terms are approximately 0.220 / 3.17 (Table 4). CEM success is identical: **5 successes out of 93** feasible val starts ($\approx 5.38\%$). ODE CEM wall-clock is substantially higher (≈ 82 min / 4859 s vs. ≈ 8 min for discrete on the same machine) because each rollout step runs four RK4 stages with four velocity evaluations each.

Item	Value
Dataset	Official HF PushT expert HDF5
Subset / seed	1000 windows / 3072
Train / val	901 / 99
CEM eval set	val-99 $\rightarrow$ 93 feasible
Encoder	ViT-tiny, patch 14, 224 ² , scratch
Predictor depth / heads	6 / 16 (AdaLN-zero)
SIGReg	$\lambda=0.09$, $M=1024$, knots 17
ODE solver	RK4, $N=4$, $\tau:0 \rightarrow 1$
Params (AR / ODE)	18,034,478 / 18,071,534
Optimizer	AdamW 5×10^{-5} , wd 10^{-3}
Train	100 epochs, bf16, grad clip 1.0
Effective batch	128 ($64 \times \text{accum } 2$)
Hardware	RTX 4060 Laptop 8 GiB

Table 2. CEM planner hyperparameters (official PushT config).

Item	Value
Solver	CEMSolver
Samples / iterations / top- k	300 / 30 / 30
Horizon / receding / action block	5 / 5 / 5
Goal offset (steps)	25
Max episode steps	100
Eval seed	42

Loss $\downarrow$ vs. CEM flat. Lower next-embedding loss does not yield higher CEM success under this budget. Several non-exclusive explanations are consistent with the data: (i) the 1k subset may be too small for planning-relevant long-horizon structure to separate the two predictors; (ii) CEM hyperparameters (horizon 5, 300 samples) may be the bottleneck rather than one-step prediction error; (iii) goal-offset feasibility and the sparse success events (only 5 hits) make the metric high-variance at $n=93$; (iv) RK4’s extra compute does not change the discrete CEM interface, so planning search explores the same action schedule. We therefore report a *tie* on CEM and a clear but modest win on loss—not a planning breakthrough.

Training dynamics. Figure 1 and Figure 2 show that both predictors decrease fit and val loss over 100 epochs on the same split. The ODE run exhibits a transiently high validation SIGReg term in early epochs (on the order of 30–100) before settling near ≈ 3.17 , comparable to the discrete baseline’s late-training SIGReg scale. This suggests that the continuous residual path does not permanently destabilize SIGReg under $\lambda=0.09$, but can delay isotropy early in training when the velocity head leaves the zero-init regime.

Protocol note. Earlier CEM probes on random full-dataset starts (e.g. 0/10 for the 100-epoch discrete checkpoint, 1/10 for a 1-epoch smoke run) are *not* val-tied and are discarded; all claims here use the val-99 / 93-feasible protocol shared with `validate/loss`. The prior 0/10 result reflected a protocol mismatch (random windows from the ~ 1.9 M pool), not total model collapse.

4.3 Limitations and future work

- **1k subset / not paper scale.** Results are on 1000 windows, not the full PushT expert corpus. Paper-scale $\approx 96\%$ success is out of scope for this study; closing that gap requires full-data 100-epoch training and paper-protocol CEM ($n=50$), which we have not run.
- **CEM tied.** Lower loss does not yield higher CEM success under this budget; planning may be bottlenecked by data scale, CEM hyperparameters, or goal-offset feasibility rather than predictor form.
- **Single environment.** Only PushT is evaluated; Cube, Two-rooms, and Reacher remain future work (official HF datasets exist but were not downloaded here).

Table 3. PushT 1k×100ep: loss and CEM (same split). Paper PushT success is under full-data training, shown for context only.

Method	Params	fit/loss	val/loss	CEM val-99
Discrete AR (LeWM)	18.03M	0.248	0.536	5/93 (5.38%)
ODE RK4 0→1 (ours)	18.07M	0.234	0.506	5/93 (5.38%)
LeWM paper (full data)	~15M	—	—	96±2.83% [†]

[†]Reported PushT success; not comparable to our 1k protocol.

Table 4. ODE run: final validation loss decomposition (epoch 100).

Metric	Value
fit/loss (train)	0.234
val/loss	0.506
val/pred ($\mathcal{L}_{\text{JEPA}}$)	≈0.220
val/sigreg	≈3.17

- **Fixed solver.** We use RK4 with $N=4$ and no adaptive stepping; Euler is implemented but not ablated at 100 epochs. Broader ODE solvers, multi-step τ schedules, and ODEWorld-style physical-time flow are unexplored.
- **Compute.** ODE CEM is $\sim 10\times$ slower than discrete AR on the same GPU; any deployment story must weigh loss gains against planning latency.
- **Single seed / single split.** All numbers use seed 3072 and one frozen 901/99 split. We do not report multi-seed confidence intervals; the 5/93 CEM count should be read as a point estimate under this protocol.

4.4 Discussion

The matched comparison supports three practical takeaways.

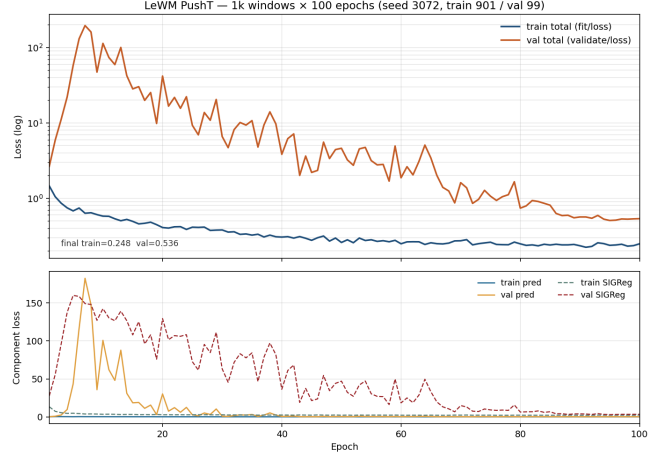
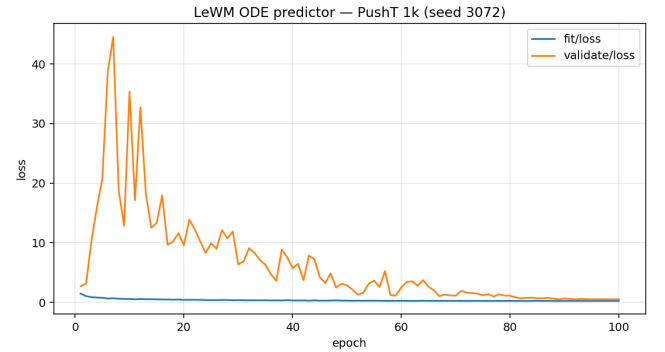
First, continuous latent dynamics are a *compatible* inductive bias for LeWM-style JEPA training: the ODE predictor trains stably for 100 epochs under the same SIGReg weight, reaches lower fit/val loss, and requires only a small parameter delta (+37,056).

Second, compatibility is not the same as a planning win. CEM success remains 5/93 for both predictors. In a low-data regime with a sparse binary success metric, improved one-step embedding fit need not rearrange which of 93 starts are solved under a fixed CEM budget. We therefore treat loss and CEM as separate axes and refuse to conflate them.

Third, protocol discipline matters as much as predictor choice. Random-start CEM probes produced misleading 0/10 or 1/10 scores that are not comparable across runs; the val-tied protocol is what makes the discrete vs. ODE comparison legitimate. Future scaling experiments should preserve the same discipline: frozen splits, shared CEM starts, and explicit separation between subset ablations and paper-scale claims.

What we are *not* claiming. We do not claim paper-scale PushT performance, multi-seed statistical significance of the 5/93 tie, superiority of ODEs for CEM in general, or that lower JEPA loss implies better control. We claim only a matched 1k comparison on official data: ODE reduces reported train/val loss and matches discrete CEM success under the val-99 protocol documented above.

Future work. Natural next steps are (1) full PushT training with the same ODE drop-in, (2) multi-environment CEM under matched splits, (3) solver ablations (N , Euler vs. RK4, adaptive integrators), (4) evaluating official HF pretrained weights under the same val-99 CEM protocol as an external reference, and (5) stronger continuous-time formulations in the spirit of ODEWorld [8] beyond a one-step RK4 map.

**Figure 1.** Discrete AR training curves on the fixed 1k split (seed 3072).**Figure 2.** ODE predictor training curves on the same 1k split. Final fit/loss ≈ 0.234 , val/loss ≈ 0.506 . Early epochs show transiently elevated val SIGReg divergence near ≈ 3.17 .

5 Conclusion

We presented a minimal continuous-time drop-in for LeWM-style JEPA world models: an action-conditioned ODE predictor whose one-step map is RK4 integration from $\tau=0$ to $\tau=1$, with encoder, SIGReg, projectors, and CEM held fixed. On official PushT data with a fixed 1k-window split (seed 3072; 901/99), the ODE model matches discrete AR CEM success (5/93) while reducing train and validation loss (0.234/0.506 vs. 0.248/0.536).

The result supports continuous latent dynamics as a viable predictor inductive bias for planning JEPA models, but—honestly—does not yet improve CEM under the 1k regime, nor close the gap to paper-scale PushT performance ($\approx 96\%$). The tied CEM outcome, despite a clear loss improvement, is itself a finding: under this budget, next-embedding fit is not a sufficient proxy for planning success. We view the study as a controlled ablation that isolates predictor form; scaling data and environments is the clear next measurement.

Natural next steps are full-dataset training, multi-environment CEM, solver ablations, and richer continuous-time formulations in the spirit of ODEWorld [8]. All numbers in this paper come from documented runs on the official Hugging Face PushT release; we did not invent metrics or synthetic episodes.

References

- [1] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023.

- [2] R. Balestriero et al. The birth of SIGReg: Sketched isotropic Gaussian regularization. *arXiv preprint*, 2023. As used in LeWM; see Maes et al. 2026.
- [3] A. Bardes, J. Ponce, and Y. LeCun. V-JEPA: Latent video prediction for self-supervised learning. *arXiv preprint arXiv:2404.08471*, 2024.
- [4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.
- [5] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *NeurIPS*, 2018.
- [6] D. Ha and J. Schmidhuber. World models. In *NeurIPS*, 2018.
- [7] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *ICML*, 2019.
- [8] D. Liu et al. ODEWorld: A continuous predictive architecture via physical-time flow. *arXiv preprint arXiv:2607.27924*, 2026.
- [9] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.